AtelierLocal - Projet final d’accessibilité

Travail en binôme · Projet sur 30 points · Délai : une semaine après la séance

Vous reprenez une application React qui présente trois ateliers culturels. Vous devez découvrir son parcours, identifier et corriger six problèmes d’accessibilité, puis ajouter un composant permettant de poser une question à l’organisateur.

L’objectif est de rendre les tâches compréhensibles et réalisables. Vous pouvez réutiliser et adapter les snippets de CinéScope. Vous n’avez pas à refaire le design, installer un serveur ou réaliser un envoi réel.

1. Votre mission et le barème

Travail	Points
Deux problèmes de parcours, à 4 points chacun	8
Quatre problèmes ciblés, à 2 points chacun	8
Un composant accessible : poser une question	10
Un mini-test sur ce composant	4
Total projet en binôme	**30**

Les deux QCM individuels valent ensemble 10 points. Pour chaque étudiant : **note finale sur 20 = (projet sur 30 + QCM sur 10) / 2**.

Les problèmes doivent être justifiés dans `AUDIT.md` et corrigés dans le code. Pour un problème ciblé : 1 point pour le constat et son impact, 1 point pour la correction. Pour un problème de parcours : 2 points pour le diagnostic et l’explication des difficultés liées, 2 points pour l’ensemble cohérent des corrections.

Si un même problème apparaît à plusieurs endroits, décrivez-le une seule fois et corrigez toutes ses occurrences.

2. Installer et démarrer

Prérequis : Node.js **24.15 ou plus récent dans la branche 24**, recommandé pour les dépendances fournies. Les branches 22 (à partir de 22.22.2) et 26 ou plus récentes sont également compatibles. Ouvrez un terminal à la racine du projet, où se trouve `package.json`.

```
npm ci
npm run dev
```

Ouvrez l’adresse affichée par Vite, habituellement `http://localhost:5173`. Dans un second terminal :

```
npm test -- --run
npm run build
```

Au départ, trois tests fonctionnels passent et le test du composant est marqué `TODO`. Cela confirme que le projet démarre ; cela ne prouve pas son accessibilité. Vous devez remplacer le

TODO par votre test.

Les ateliers et les actions sont fictifs. Employez uniquement des noms et adresses électroniques fictifs. Aucun paiement, email ni réservation réelle n’est réalisé.

3. Partie 1 - Auditer et corriger l’application

Explorez AtelierLocal et ses principales fonctionnalités : rechercher un atelier, consulter ses détails et effectuer une préinscription. Identifiez, documentez et corrigez **six problèmes d’accessibilité**.

Démarche attendue

1. **Découvrir le parcours** : utilisez les fonctionnalités et observez les informations présentées ainsi que les changements provoqués par vos actions.
2. **Parcourir l’application au clavier** : essayez les commandes, suivez le focus et vérifiez que les interactions sont réalisables.
3. **Vérifier la sémantique HTML** : examinez la structure de la page et le choix des éléments employés.
4. **Examiner les informations et les états** : vérifiez que les informations nécessaires sont compréhensibles et que les actions et leurs résultats sont cohérents.
5. **Approfondir votre diagnostic** : vous pouvez vous appuyer sur les outils de diagnostic présentés en cours pour compléter vos vérifications manuelles.
6. **Documenter, corriger et vérifier** : expliquez les problèmes dans `AUDIT.md`, appliquez vos corrections, puis rejouez les manipulations pour vérifier le résultat.

Ne vous limitez pas à la page au chargement : observez également ce qui change lorsque vous utilisez ses fonctionnalités.

Vos corrections doivent préserver le fonctionnement de l’application.

Document à produire : AUDIT.md

Pour chacun des six problèmes, utilisez les quatre rubriques suivantes :

- **Problème identifié** : le défaut observé et la difficulté qu’il peut provoquer pour une personne.
- **Reproduction** : les étapes permettant de constater le problème, en précisant si nécessaire les outils employés.
- **Comportement attendu** : ce qu’une personne devrait pouvoir comprendre ou réaliser.
- **Correction à apporter** : la modification envisagée et sa justification. Après réalisation, précisez la modification réellement effectuée et le résultat de votre vérification.

Quelques phrases par rubrique suffisent. Vous pouvez partir de `MODELE-AUDIT.md` ou organiser vous-mêmes votre fichier autour de ces quatre rubriques. Une liste d’attributs ajoutés sans explication ne suffit pas.

4. Partie 2 - Créer un composant : poser une question

Une personne souhaite obtenir une information avant de s’inscrire à un atelier. Ajoutez une action **« Poser une question »** sur chaque carte. Elle ouvre une fenêtre modale permettant de contacter l’organisateur au sujet de cet atelier.

Contenu et fonctionnement attendus

Le composant doit comporter :

- un titre permettant d’identifier l’atelier concerné ;
- un **select natif** pour choisir le sujet : « Les horaires », « Le matériel à apporter », « L’accès au lieu » ou « Un autre sujet » ;
- un champ pour l’adresse électronique ;

- une zone de texte pour rédiger la question ;
- une action pour envoyer la demande ;
- une possibilité d'annuler ou de fermer la fenêtre.

Pour mettre en place le sélecteur, consultez la documentation MDN de l'élément `<select>`. Utilisez le contrôle HTML natif : il n'est pas demandé de reconstruire une liste déroulante personnalisée.

Si vous ne parvenez pas à intégrer le select, vous pouvez proposer un groupe de boutons radio présentant les mêmes sujets. Expliquez ce choix dans votre compte rendu. Dans les deux cas, le choix doit être accessible au clavier.

Les trois champs sont obligatoires. Aucun sujet ne doit être présélectionné : le choix initial du select invite à sélectionner un sujet ; avec des boutons radio, aucun n'est coché au départ. L'adresse doit être non vide et sa syntaxe vérifiée ; il n'est pas demandé de vérifier son existence. La question ne peut pas être vide ou constituée uniquement d'espaces.

Après un envoi valide, la fenêtre se ferme et une confirmation indique **l'atelier concerné et le sujet choisi**, sans reprendre les coordonnées de contact. Annuler ou fermer ne doit transmettre aucune demande. Contacter l'organisateur reste possible lorsqu'un atelier est complet.

Accessibilité du parcours

La fonctionnalité doit permettre de **comprendre quoi renseigner, identifier une éventuelle difficulté, la corriger et connaître le résultat de sa demande**.

À vous de déterminer et de mettre en œuvre les solutions d'accessibilité adaptées, en vous appuyant sur les séances précédentes. Cette attente concerne l'ensemble du parcours, de l'ouverture à la fermeture, y compris les tentatives d'envoi qui n'aboutissent pas. Vos choix HTML, la navigation au clavier et la manière de transmettre les informations font partie du travail évalué.

Ajoutez une courte section **« Composant créé »** dans `AUDIT.md` pour expliquer vos principaux choix et les manipulations réalisées pour les vérifier.

Ressources fournies

Les types, les options et la fonction `readRequest(form, kind, workshopId)` sont fournis dans `src/requests.ts`. Appelez cette fonction avec `kind = "question"`. Elle utilise les noms de champs **choice**, **contact** et **message**, et retourne `{ values, errors }`, comme le validateur de CinéScope. Elle n'affiche rien et ne prend pas en charge l'interface.

Le contrat `RequestDialogProps` fournit l'atelier concerné et les fonctions `onSubmit(values)` et `onClose()`. `WorkshopCard` possède une propriété facultative `extraAction` pour intégrer votre bouton. Les styles de base sont disponibles dans `src/styles.css`.

Vous pouvez réutiliser et adapter les snippets des cours précédents. Aucun email réel ni envoi réseau n'est attendu : l'envoi est simulé dans l'application. Utilisez uniquement des coordonnées fictives.

Barème du composant - 10 points

Critère	Points
Intégration et prise en compte du bon atelier	2
Accessibilité de la modale et continuité de la navigation	3
Accessibilité du formulaire et possibilité de se reprendre après une erreur	3
Confirmation compréhensible et cohérence des actions	2

5. Écrire le mini-test - 4 points

Remplacez le TODO de `src/__tests__/Composant.test.tsx` par un test suivant la méthode utilisée en séance 3 : affichez l'application avec `render(<App />)`, puis simulez les actions avec `userEvent`. Vitest et Testing Library sont déjà configurés.

Le scénario à tester est le suivant :

1. Ouvrez « Poser une question » pour l'atelier **« Une tasse en céramique »**.
2. Choisissez **« Le matériel à apporter »** et rédigez une question. Si vous avez utilisé des boutons radio, sélectionnez ce même sujet.
3. Laissez l'adresse électronique vide, puis essayez d'envoyer.
4. Vérifiez qu'une erreur est indiquée, que la modale reste ouverte et que le sujet choisi et la question sont conservés. Aucune confirmation d'envoi ne doit apparaître.
5. Renseignez une adresse électronique valide, puis envoyez à nouveau.
6. Vérifiez que la modale se ferme et qu'une confirmation indique **« Une tasse en céramique »** et **« Le matériel à apporter »**.

Barème : ouverture et saisie du scénario (1 point), erreur et absence de confirmation après l'envoi incomplet (1 point), conservation du sujet et de la question (1 point), fermeture et confirmation du bon atelier et du bon sujet après l'envoi valide (1 point).

6. Rendu et vérifications finales

Un seul dépôt GitHub accessible à l'enseignant par binôme. Transmettez son lien selon le canal indiqué en cours. Complétez le README fourni avec les noms des membres du binôme. Mettez à jour les commandes de lancement si vous les avez modifiées. Le délai est d'une semaine après la séance ; la date et l'heure exactes seront annoncées par l'enseignant.

Le dépôt doit contenir :

- l'application et ses corrections, avec `package.json` et `package-lock.json` ;
- le nouveau composant intégré ;
- votre mini-test ;
- `AUDIT.md`, avec les six problèmes et leurs corrections ;
- le README fourni, complété avec les noms des membres du binôme et les commandes de lancement à jour.

N'incluez pas `node_modules` ni `dist`. Avant de rendre, lancez `npm test -- --run` et `npm run build`, puis refaites vos essais sur l'application et votre nouvelle action au clavier. Si vous avez utilisé un lecteur d'écran, indiquez lequel et ce que vous avez réellement constaté. Un test DOM ne prouve pas la restitution vocale.

Le projet doit garder la recherche, les données et les fonctions utiles de l'application. Les modifications visuelles nécessaires à l'accessibilité sont autorisées ; une refonte graphique n'apporte pas de points en elle-même.